Chapter 1. 헬로 파이썬

1. cmd 를 통한 파일 실행

명령 프롬프트

```
Microsoft Windows [Version 10.0.18363.1198]
(c) 2019 Microsoft Corporation. All rights reserved.

C:₩Users₩82107>cd Desktop₩AI₩밑바닥부터 시작하는 딥러닝

C:₩Users₩82107₩Desktop₩AI₩밑바닥부터 시작하는 딥러닝>python pa1.py
hi
```

- 저장된 디렉토리로 이동하여 파이썬을 실행할수 있다.

2. 넘파이

- 딥러닝을 구현하다 보면 배열이나 행렬 계산이 많이 등장하는데 이때 넘파이를 이용하면 편리

```
x=[1,2,3]
y=[3,4,5]
a=x-y
print(a)
```

넘파이를 사용안할경우 다음과 같은 에러발생

```
a=x-y
TypeError: unsupported operand type(s) for -: 'list' and 'list'
```

```
import numpy as np

x=np.array([1,2,3])
y=np.array([3,4,5])
a=x-y
print(a)
```

넘파이를 이용하면 배열간 연산이 쉬워진다.

```
[-2 -2 -2]
```

```
import numpy as np

x=np.array([1.0,2.0,3.0])
y=np.array([2.0,4.0,6.0])
print(x+y)
print(x-y)
print(x*y)
print(x/y)
print(x/2.0)
```

```
[3. 6. 9.]
[-1. -2. -3.]
[ 2.  8. 18.]
[0.5 0.5 0.5]
[0.5 1.  1.5]
```

- 넘파이의 N차원 배열

```python
import numpy as np

A=np.array([[1,2],[3,4]])
print(A)
print(A.shape,A.dtype)

B=np.array([[3,0],[0,6]])
print(A+B)
print(A*B)
print(np.dot(A,B))
print(np.matmul(A,B))
# *는 요소끼리의 곱이다.
# dot 내적 , matmul 행렬의 곱
print(A*10)
```

```
[[1 2]
 [3 4]]
(2, 2) int32
[[ 4  2]
 [ 3 10]]
[[ 3  0]
 [ 0 24]]
[[ 3 12]
 [ 9 24]]
[[ 3 12]
 [ 9 24]]
[[10 20]
 [30 40]]
```

- 브로드캐스트

형상이 다른 배열끼리 곱할때 있는데 배열을 똑같은 형상으로 맞춰주는 똑똑한 기능을 브로드캐스트라고 한다.

```python
import numpy as np

A=np.array([[1,2],[3,4]])
B=np.array([10,20])
print(A*B)
# B차원을 [10,20] --> A에 맞춰 [[10,20],[10,20]] 으로 바꿔준다.
# 브로드캐스트
```

```
[[10 40]
 [30 80]]
```

- 원소접근

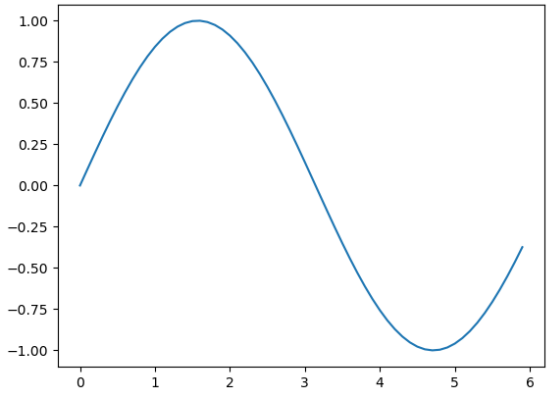
```python
import numpy as np

X=np.array([[51,55],[14,19],[0,4]])
print(X)
print(X[0])
print(X[0][1])

for row in X:
    print(row)
X=X.flatten()
# x를 1차원 배열로 변환(평탄화)
print(X)
print(X[np.array([0,2,4])])
# 인덱스가 0,2,4인 원소 얻기
print(X)
# 원래 X는 유지된다.
```

```
[[51 55]
 [14 19]
 [ 0  4]]
[51 55]
55
[51 55]
[14 19]
[0 4]
[51 55 14 19  0  4]
[51 14  0]
[51 55 14 19  0  4]
```

3. matplotlib

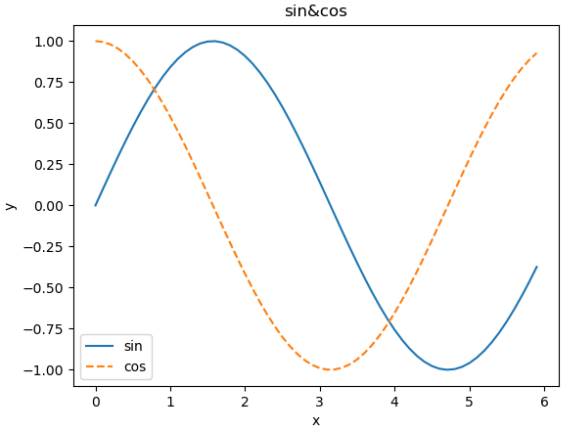
- 그래프 그리기

```python
import numpy as np
import matplotlib.pyplot as plt

x=np.arange(0,6,0.1)
#0 에서 6까지 0.1간격으로 생성
y=np.sin(x)

plt.plot(x,y)
plt.show()
```



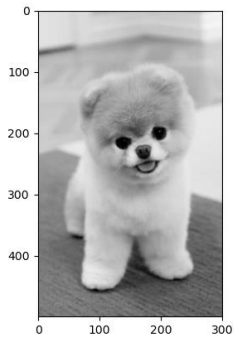
np.arange(시작,끝,간격) --> [0,0.1,0.2,....,5.8,5.9] 라는 데이터 형성

```python
import numpy as np
import matplotlib.pyplot as plt

x=np.arange(0,6,0.1)
y1=np.sin(x)
y2=np.cos(x)

plt.plot(x,y1,label='sin')
plt.plot(x,y2,linestyle="--",label='cos')
plt.xlabel('x')
plt.ylabel('y')
plt.title("sin&cos")
plt.legend()
plt.show()
```



- 이미지 표시하기

```python
import matplotlib.pyplot as plt
from matplotlib.image import imread

img=imread('gray.jpg')

plt.imshow(img)
plt.show()
```



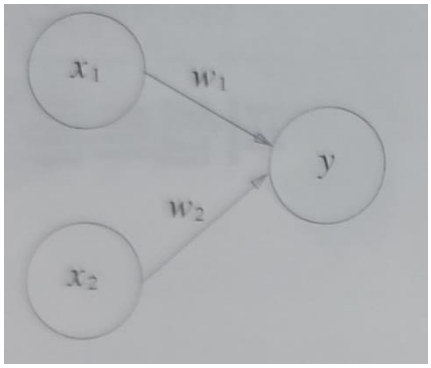
Chapter 2. 퍼셉트론

1. 퍼셉트론이란 ?

- 퍼셉트론은 다수의 신호를 입력 받아 하나의 신호를 출력

(1 : 신호가 흐른다. 0:신호가 흐르지 않는다.)

(입력으로 2개의 신호를 받은 퍼셉트론)



$$y = \begin{cases} 0 \ (w_1x_1 + w_2x_2 \le \theta) \\ 1 \ (w_1x_1 + w_2x_2 > \theta) \end{cases}$$

(x1,x2 : 입력신호 , y : 출력신호 , w1,w2 : 가중치)

(그림의 원 : 뉴런 혹은 노드 라고 부른다.)

(뉴런에서 보내온 신호의 총합이 정해진 한계(임계값)(θ)를 넘어설 때만 1을 출력한다)

(이를 뉴런이 활성화한다 라 표현하기도 한다.)

(w1,w2 이런 가중치는 전류에서 말하는 저항에 해당한다. 전류에서는 저항이 낮을수록 큰 전류가 흐르지만 퍼셉트론의 가중치는 그값이 클수록 강한 신호를 흘려보낸다. 서로 작용하는 방향은 반대지만 신호가 얼마나 잘 흐르는가를 통제한다는 점에서 저항과 가중치는 같은기능을 한다.)

2. 단순한 논리 회로

그림 2-2 AND 게이트의 진리표

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

그림 2-3 NAND 게이트의 진리표

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

그림 2-4 OR 게이트의 진리표

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

y=w1x1+w2x2 , y와 θ값을 비교하여 출력하는 값 조정.

게이트를 표현할 수 있는 각각의 x1, x2,y값에 따른 w1과 w2 θ 매겨변수를 찾는것이다.

학습 : 적절한 매개변수 값을 정하는 작업(사람은 퍼셉트론의 구조(모델)를 고민하고 컴퓨터에 학습할 데이터를 주는 일을 한다.

```python
def AND(x1,x2):
    w1,w2,theta=0.5,0.5,0.7
    tmp=x1*w1+x2*w2
    if tmp<=theta:
        return 0
    elif tmp>theta:
        return 1

print(AND(0,0))
print(AND(1,0))
print(AND(0,1))
print(AND(1,1))
# AND 게이트를 만드는 매개변수예 w1,w2,theta=0.5,0.5,0.7
```

```
0
0
0
1
```

- 가중치와 편향 도입

θ를 -b 로 치환하면 퍼셉트론의 동작이 다음 식처럼 된다.

$$y = \begin{cases} 0 \ (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 \ (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

b: 편향(bias)

w1,w2 : 가중치(weight)

0을 넘으면 1을 출력 그렇지않으면 0 출력

- AND 게이트 , NAND 게이트 , OR 게이트 구현

(가중치와 편향값은 게이트에 맞는 값을 찾은 것이다. 실제로는 학습을 통하여 적절한 값을 찾는다.)

```python
import numpy as np

def AND(x1,x2):
    x=np.array([x1,x2])
    w=np.array([0.5,0.5])
    b=-0.7
    tmp=np.sum(w*x)+b
    if tmp <=0:
        return 0
    else:
        return 1
"""
 b가 -0.1 이면 각 입력신호에 가중치를 곱한 값들의 합이 0.1을 초과할 때만 뉴런이 활성화
 b가 -20.0 이면 20을 넘지않으면 뉴런이 활성화되지 않는다.
 편향이란 용어는 한쪽으로 치우쳐 균형을 깬다는 의미
 편향의 값은 뉴련이 얼마나 쉽게 활성화되는지를 결정한다.
"""
```

```python
def NAND(x1,x2):
    x=np.array([x1,x2])
    w=np.array([-0.5,-0.5])
    b=0.7
    tmp=np.sum(w*x)+b
    if tmp<=0:
        return 0
    else:
        return 1
```

```python
def OR(x1,x2):
    x=np.array([x1,x2])
    w=np.array([0.5,0.5])
    b=-0.2
    tmp=np.sum(w*x)+b
    if tmp<=0:
        return 0
    else:
        return 1
```

3. 퍼셉트론의 한계

- XOR 게이트(베타적 논리합 이라는 논리 회로 x1과 x2 중 한쪽만 1일경우 1을 출력)

그림 2-5 XOR 게이트의 진리표

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

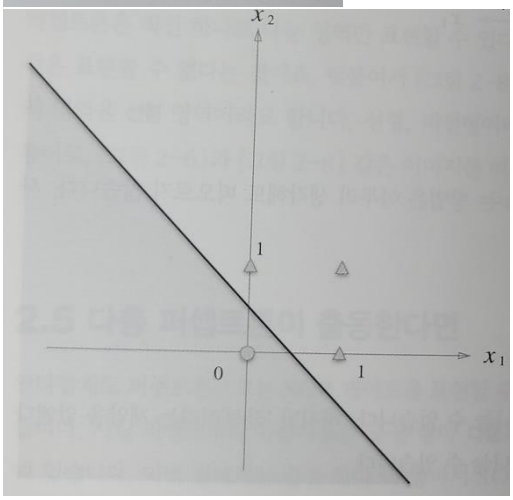
지금까지 본 퍼셉트론으로는 이 XOR 게이트를 구현할수 없다.

(퍼셉트론의 시각화를 통하여 그 이유 설명)

- 퍼셉트론의 시각화 (회색영역은 y 값 0 출력) (OR 을 예로 들었다.)

$$y = \begin{cases} 0 & (-0.5 + x_1 + x_2 \le 0) \\ 1 & (-0.5 + x_1 + x_2 > 0) \end{cases}$$

다음 식의 직선의 방정식을 그려보면 다음과 같이 표현되게 된다.

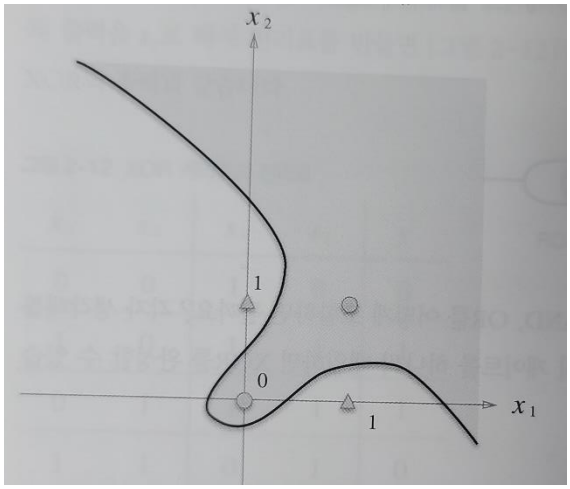


x2=-x1+0.5 식

x2가 -x1+0.5 보다 크다면 1을 출력

x2가 -x1+0.5 보다 작으면 0을 출력

- XOR 을 시각화 한경우



다음과 같이 0을 표시하는 동그라미 표시와

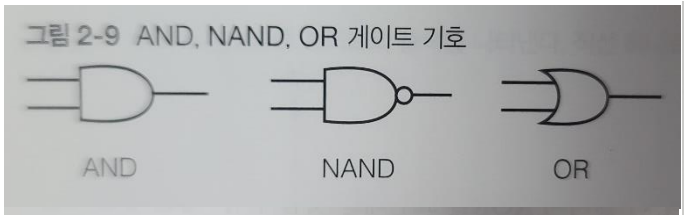
1을 표시하는 삼각형 표시를 절대로 한개의 직선으로는 나누지 못한다.

왼쪽 그림과 같이 곡선으로 나누면 나누는 것이 가능하다.

이처럼 퍼셉트론은 직선 하나로 나눈 영역만 표현할 수 있다는 한계가 있다.

4. 한계를 극복하는 방법

- 퍼셉트론으로는 XOR 게이트를 표현할 수 없다. 하지만 퍼셉트론을 층으로 쌓아 다층 퍼셉트론(multi-layer perceptron)을 만든다면 가능하다.



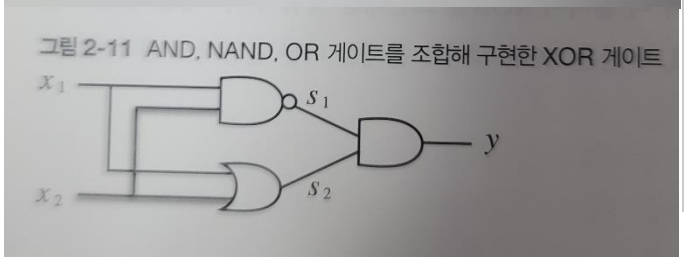


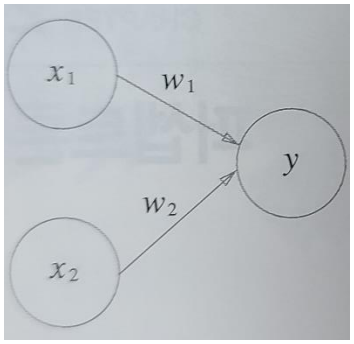
그림 2-12 XOR 게이트의 진리표

x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

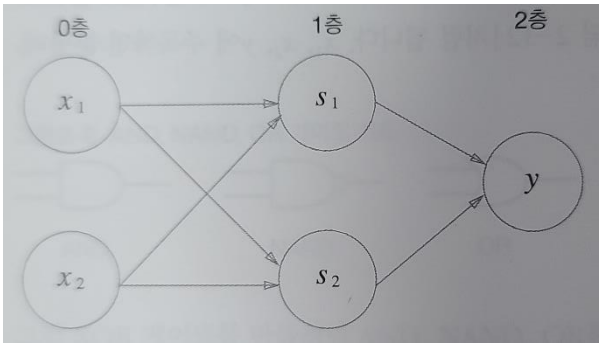
x1과x2는 NAND 와 OR게이트의 입력이 되고 NAND와 OR의 출력이 AND 게이트의 입력으로 이어진다. (s1 은 NAND 의 결과 ,s2 는 OR 의 결과 , y는 s1과 s2의 AND의 결과다)

```
def XOR(x1,x2):
    s1=NAND(x1,x2)
    s2=OR(x1,x2)
    y=AND(s1,s2)
    return y

print(XOR(0,0))      0
print(XOR(1,0))      1
print(XOR(0,1))      1
print(XOR(1,1))      0
```

(단층 퍼셉트론)



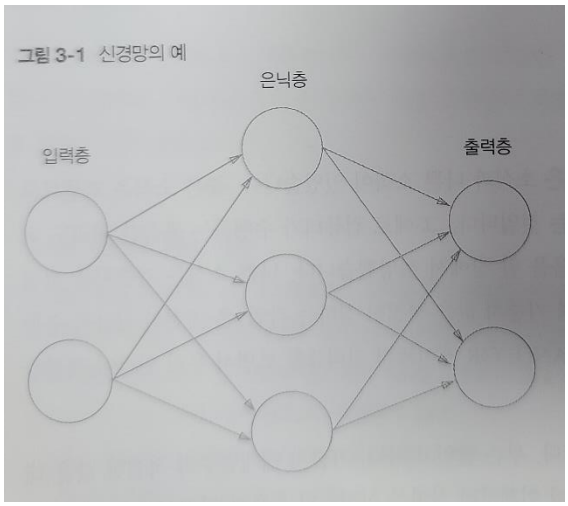
(2층 퍼셉트론)(다층 퍼셉트론)



AND,OR,NAND가 단층 퍼셉트론인데 XOR 은 2층 퍼셉트론이다 . 이처럼 단층 퍼셉트론으로는 표현하지 못한 것을 층을 하나 늘려 구현할수 있다.

Chapter 3. 신경망

1. 퍼셉트론에서 신경망으로



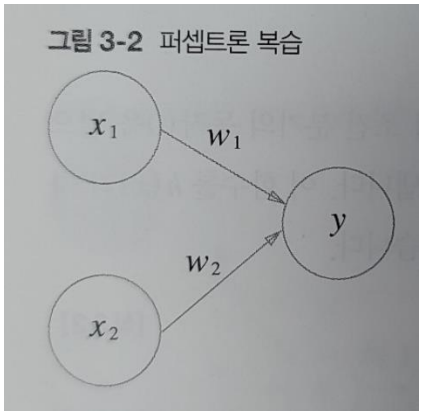
0층 : 입력층

1층 : 은닉층

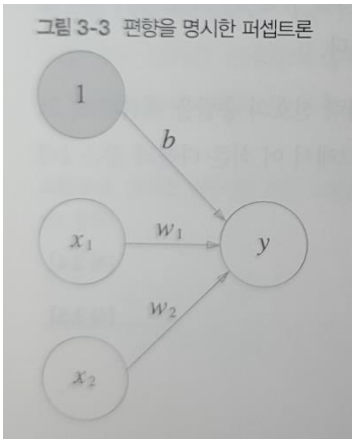
2층 : 출력층

2층 신경망

(편향을 표시하지 않은 퍼셉트론)



(편향을 명시한 퍼셉트론)



퍼셉트론을 수식으로 나타내면 다음과 같다.

$$y = \begin{cases} 0 \ (b + w_1x_1 + w_2x_2 \le 0) \\ 1 \ (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

이 식을 더 간단한 형태로 나타내면 다음과 같이 나타나게된다.

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 \ (x \le 0) \\ 1 \ (x > 0) \end{cases}$$

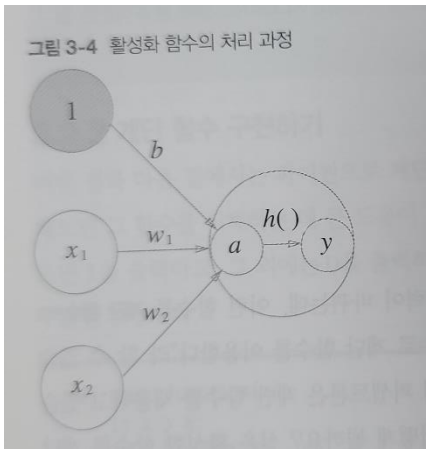
a=b+w1x1+w2x2

y=h(a)

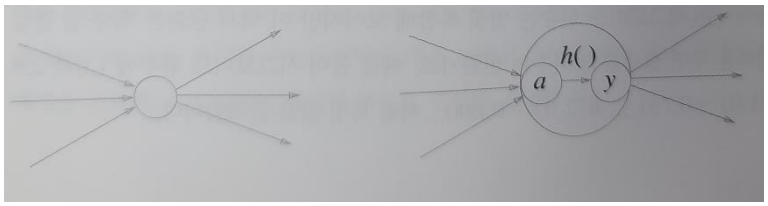
로 표현 가능하다.

- 활성화 함수의 등장(activation function)

활성화 함수는 입력 신호의 총합이 활성화를 일으키는지를 정하는 역할을 한다.



왼쪽은 일반적인 뉴런, 오른쪽은 활성화 처리 과정을 명시한 뉴런(a는 입력신호의 총합 , h()는 활성화 함수, y 는 출력)



2. 활성화 함수

- 계단 함수(입력이 0을 넘으면 1을 출력, 그 외에는 0을 출력)

```python
import numpy as np

def step_function(x):
    if x>0:
        return 1
    else:
        return 0
# 이 구현은 넘파이 배열을 인수로 넣을 수없다. 오로지 실수 만가능
```

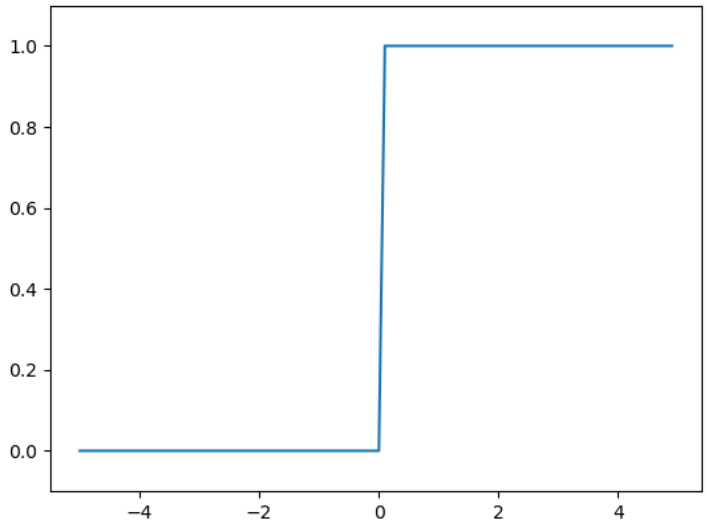
```python
def step_function1(x):
    y=x>0
    return y.astype(np.int)
# 배열을 인수로 넣을수 있다.
x=np.array([-1,1,2])
print(x)
y=x>0
print(y)
print(y.astype(np.int))
# 넘파이 배열의 자료형을 변환할 때는 astype() 메서드 사용
# bool 을 int 로 변환하면 True 는 1로 False 는 0으로 변환된다.

print(step_function1(x))
```

```
[-1  1  2]
[False  True  True]
[0 1 1]
[0 1 1]
```

계단함수 그래프 출력

```python
import numpy as np
import matplotlib.pyplot as plt
def step_function(x):
    return np.array(x>0,dtype=np.int)

x=np.arange(-5.0,5.0,0.1)
y=step_function(x)
plt.plot(x,y)
plt.ylim(-0.1,1.1)
# y축 범위 지정
plt.show()
```

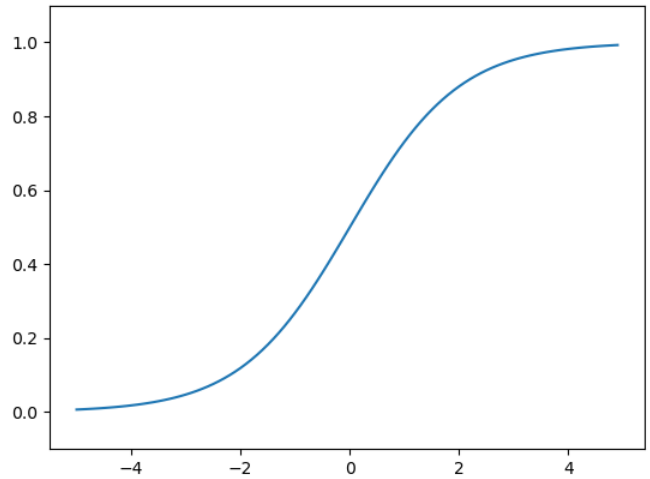
함수를 왼쪽 처럼 위의 설명을 한 번에 구현 가능하다.



- 시그모이드 함수

```python
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1/(1+np.exp(-x))
# np로 해주는 이유 x 값에 배열이 들어와도 실행될수 있도록 하기 위함이다.

x=np.array([-1,1,2])
print(sigmoid(x))

x=np.arange(-5,5,0.1)
y=sigmoid(x)
plt.plot(x,y)
plt.ylim(-0.1,1.1)
plt.show()
```

```
[0.26894142 0.73105858 0.88079708]
```



- 계단함수와 시그모이드 함수의 차이점

시그모이드는 부드러운 곡선이며 입력에 따라 출력이 연속적으로 변화한다. 한편, 계단 함수는 0을 경계로 출력이 갑자기 바뀌어 버린다. 시그모이드 함수의 이 매끈함이 신경망 학습에서 아주 중요한 역할을 하게 된다. (둘다 비선형 함수이다. 선형함수는 직선한개로 표현되는것)

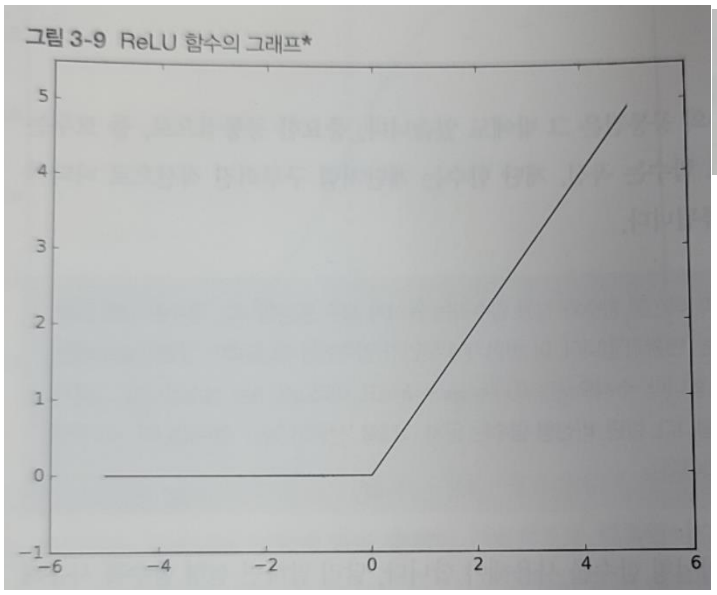
신경망에서는 활성화 함수로 비선형 함수를 사용해야 한다. 달리 말하면 선형함수를 사용해서는 안된다. 선형 함수를 이용하면 신경망의 층을 깊게 하는 의미가 없어진다.

h(x)=cx --> y(x)=h(h(h(x))) 가되는데 y(x)=c^3x 이므로 실은 y(x)=ax 랑 똑같은것이다(a=c^3)

즉 은닉층이 없어도 표현 가능 선형 함수를 이용해서는 여러 층으로 구성하는 이점을 살릴 수가 없다.

- ReLU 함수

입력이 0을 넘으면 그 입력 그대로를 출력하고 0이하면 0을 출력하는 함수



$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

```
def relu(x):
    return np.maximum(0,x)
```

간단한 함수로 ReLU 구현가능

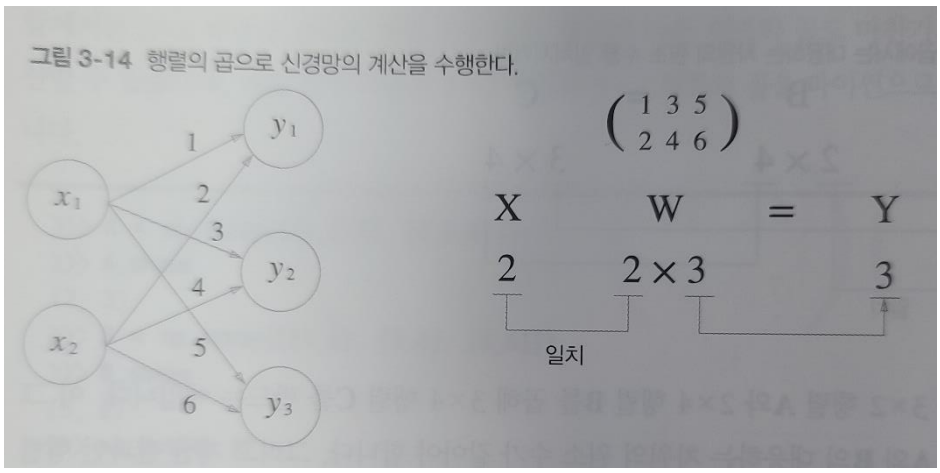
3. 다차원 배열의 계산

```
import numpy as np

A=np.array([1,2,3,4])
print(A)
print(np.ndim(A))
# []의 개수라고 생각
print(A.shape)
print(A.shape[0])

B=np.array([[1,2],[3,4],[5,6]])
print(B)
print(np.ndim(B))
print(B.shape)
```

```
[1 2 3 4]
1
(4,)
4
[[1 2]
 [3 4]
 [5 6]]
2
(3, 2)
```

np.ndim 은 [] 의 개수라고 생각 , shape 에서 제일 안쪽 괄호의 요소개수가 (,) 오른쪽의 값이다.

- 신경망에서의 행렬 곱




```python
import numpy as np

X=np.array([1,2]) # 1x2
W=np.array([[1,3,5],[2,4,6]]) # 2x3
Y=np.dot(X,W) # 1x3
# 신경망에서 행렬의 곱을 할때 X가 앞에 있음을 주의하자 !
print(Y)
```

```
[ 5 11 17]
```

4. 3층 신경망 구현하기

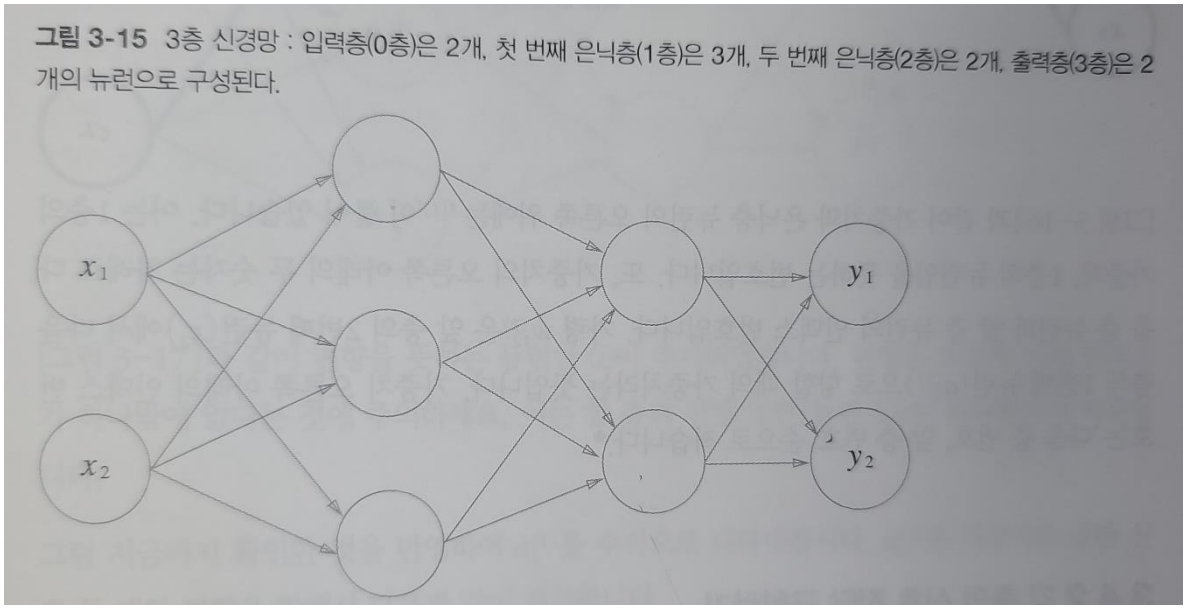


그림 3-16 중요한 표기

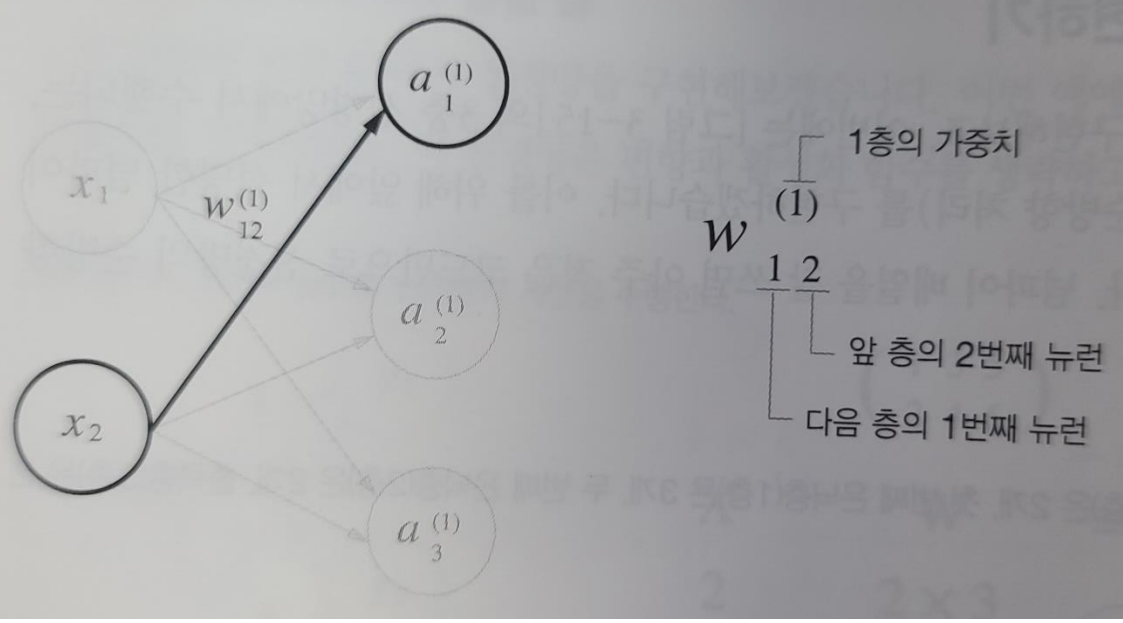
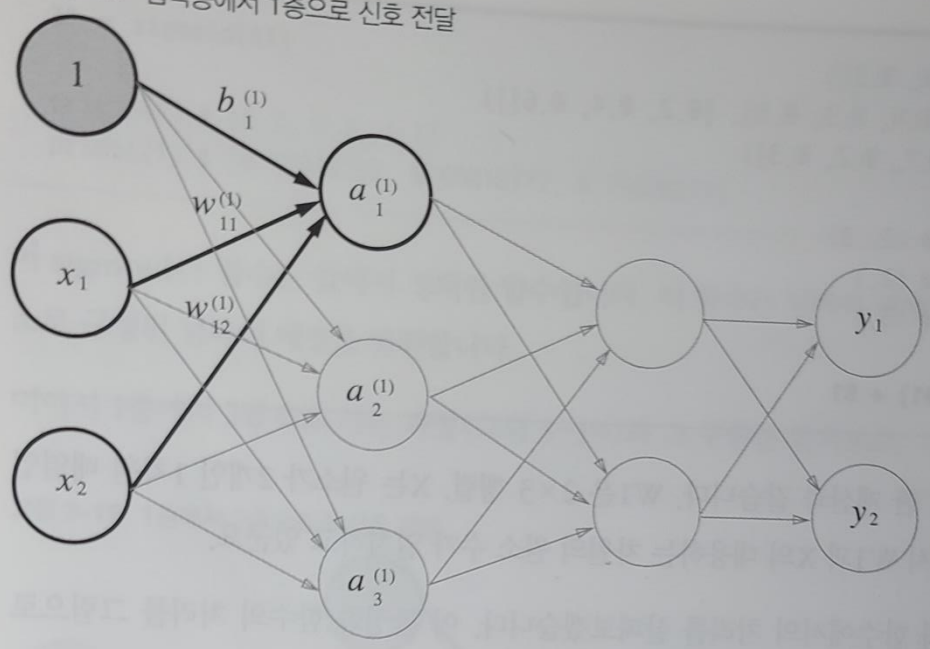


그림 3-17 입력층에서 1층으로 신호 전달



$$a_1^{(1)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)} \quad \text{[식 3.8]}$$

여기에서 행렬의 곱을 이용하면 1층의 '가중치 부분'을 다음 식처럼 간소화할 수 있습니다.

$$\mathbf{A}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{B}^{(1)} \quad \text{[식 3.9]}$$

이때 행렬 $\mathbf{A}^{(1)}$, $\mathbf{X}$, $\mathbf{B}^{(1)}$, $\mathbf{W}^{(1)}$은 각각 다음과 같습니다.

$$\mathbf{A}^{(1)} = \left(a_1^{(1)} \;\; a_2^{(1)} \;\; a_3^{(1)}\right),\ \mathbf{X} = \left(x_1 \;\; x_2\right),\ \mathbf{B}^{(1)} = \left(b_1^{(1)} \;\; b_2^{(1)} \;\; b_3^{(1)}\right)$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

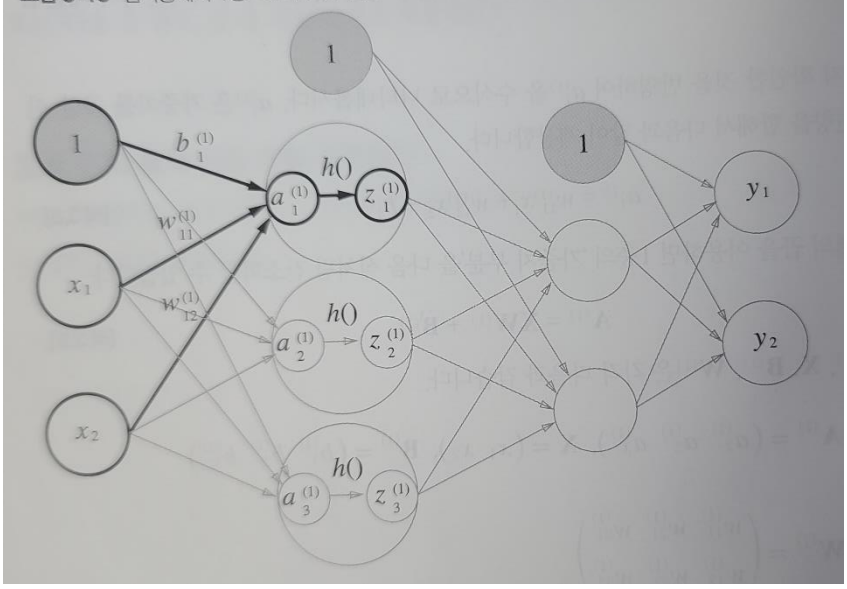
W행렬 행의 개수=x의 개수(bias를 뺀 현재층 노드의 개수) , 열의 개수=다음층 노드의 개수

```python
import numpy as np

X=np.array([1.0,0.5])  # 1x2
W1=np.array([[0.1,0.3,0.5],[0.2,0.4,0.6]])  # 2x3
B1=np.array([0.1,0.2,0.3]) # 1x3

print(W1.shape)
print(X.shape)
print(B1.shape)

A1=np.dot(X,W1)+B1
print(A1)
```

가중치와 편향값은 처음에 임의로 잡은것

```
(2, 3)
(2,)
(3,)
[0.3 0.7 1.1]
```

그림 3-18 입력층에서 1층으로의 신호 전달




```python
def sigmoid(x):
    return 1/(1+np.exp(-x))

Z1=sigmoid(A1)
print(Z1)
```

```
[0.3 0.7 1.1]
[0.57444252 0.66818777 0.75026011]
```

A1 인자가 모두 0보다 크므로 Z1 값이 모두 0.5보다 큰값이 나왔다.

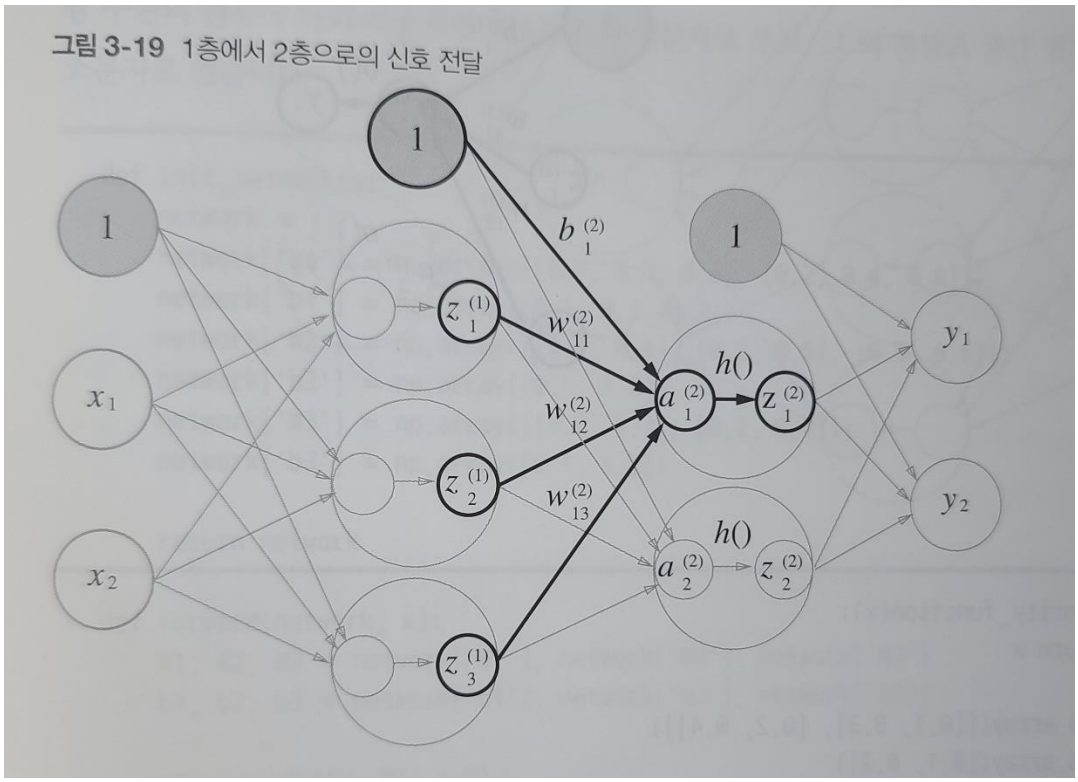


그림 3-19 1층에서 2층으로의 신호 전달

```python
W2=np.array([[0.1,0.4],[0.2,0.5],[0.3,0.6]])
B2=np.array([0.1,0.2])

A2=np.dot(Z1,W2)+B2
print(A2)

Z2=sigmoid(A2)
print(Z2)
```

```
[0.3 0.7 1.1]
[0.57444252 0.66818777 0.75026011]
[0.51615984 1.21402696]
[0.62624937 0.7710107 ]
```

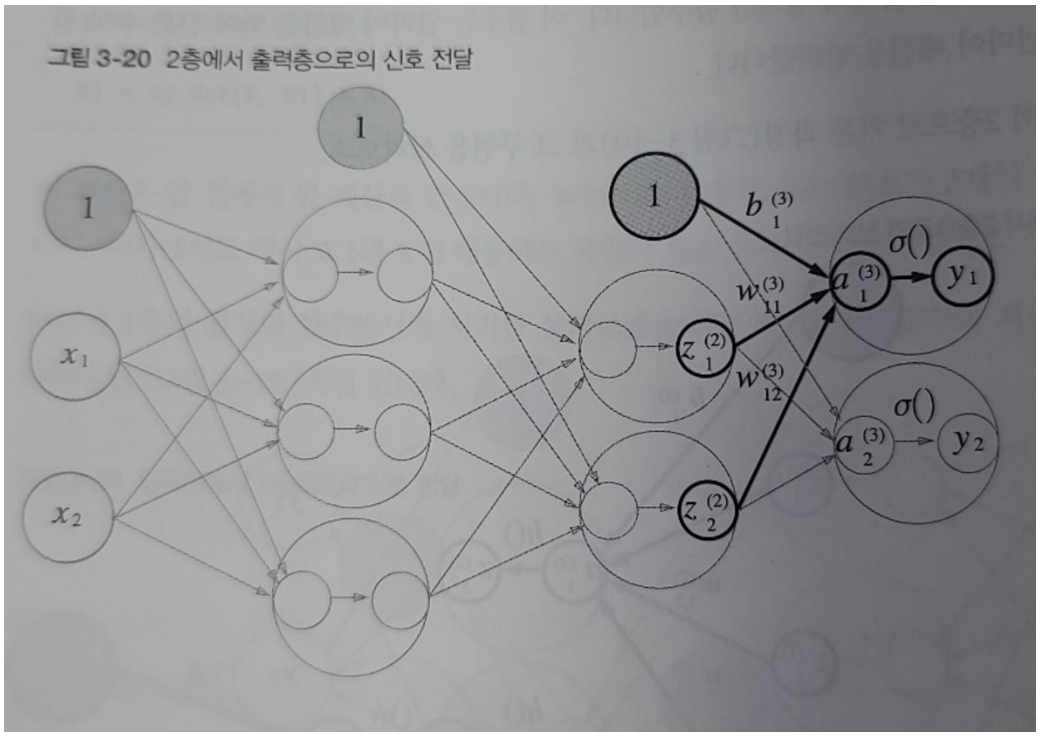


그림 3-20 2층에서 출력층으로의 신호 전달

```python
def identity_function(x):
    return x

W3=np.array([[0.1,0.3],[0.2,0.4]])
B3=np.array([0.1,0.2])

A3=np.dot(Z2,W3)+B3
print(A3)
Y=identity_function(A3)
print(Y)
# 항등 함수 identity_function()은 입력을 그대로 출력하는 함수이다.
# 굳이 정의할 필요는 없지만 그동안의 흐름과 통일하기 위해 이렇게 구현했다.
```

```
[0.3 0.7 1.1]
[0.57444252 0.66818777 0.75026011]
[0.51615984 1.21402696]
[0.62624937 0.7710107 ]
[0.31682708 0.69627909]
[0.31682708 0.69627909]
```

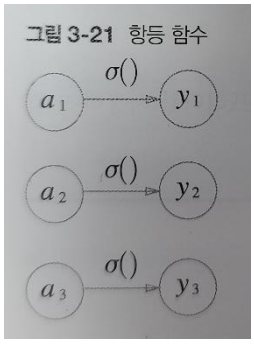
출력층의 활성화 함수는 은닉층의 활성화 함수 h()와 다르게 σ()로 표시하였다 (시그마)

출력층의 활성화함수는 풀고자 하는 문제의 성질에 맞게 정한다. 예를 들어 회귀에는 항등 함수를 , 2클래스 분류에는 시그모이드 함수를 다중 클래스 분류에는 소프트맥스 함수를 사용하는 것이 일반적이다.

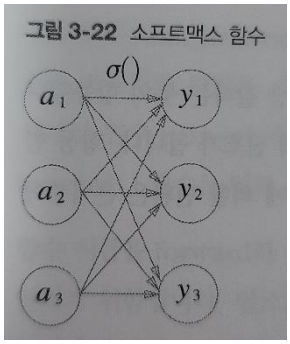
5. 출력층 설계하기

신경망은 분류와 회귀 모두에 이용할 수 있다. 다만 둘 중 어떤 문제냐에 따라 출력층에서 사용하는 활성화 함수가 달라진다. 일반적으로 회귀에는 항등 함수를 분류에는 소프트맥스함수 사용

- 항등 함수와 소프트맥스 함수 구현하기



항등함수(identity function)는 입력을 그대로 출력한다.



소프트맥스 함수(softmax function)의 식을 나타내면 다음과 같다.

yk를 다 더하게 되면 1이 된다.

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n} \exp(a_i)}$$

n은 출력층의 뉴런 수

yk는 그중 k 번째 출력

- 소프트맥스 함수 구현

```python
import numpy as np

a=np.array([0.3,2.9,4.0])
exp_a=np.exp(a)
print(exp_a)

sum_exp_a=np.sum(exp_a)
print(sum_exp_a)

y=exp_a/sum_exp_a
print(y)
print(sum(y))
```

```
[ 1.34985881 18.17414537 54.59815003]
74.1221542101633
[0.01821127 0.24519181 0.73659691]
1.0
```

함수로 구현할 경우

```python
def softmax(a):
    exp_a=np.exp(a)
    sum_exp_a=np.sum(exp_a)
    y=exp_a/sum_exp_a

    return y
```

- 소프트맥스 함수 구현시 주의할점

지수 함수를 사용하여 지수가 조금만 커져도 무한대를 나타내는 inf가 되어버린다.(오버플로)

이 문제를 해결하기 위하여 다음과 같은 방법을 사용한다.

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n}\exp(a_i)} = \frac{C\exp(a_k)}{C\sum_{i=1}^{n}\exp(a_i)} = \frac{\exp(a_k+\log C)}{\sum_{i=1}^{n}\exp(a_i+\log C)} = \frac{\exp(a_k+C')}{\sum_{i=1}^{n}\exp(a_i+C')}$$

똑같은 값을 분모 분자에 곱해주는 것이므로 상관없다.

일반적으로 C′ 값은 오버플로를 막을 목적으로는 입력 신호 중 최댓값을 이용하는 것이 일반적이다.

```python
import numpy as np

a=np.array([1010,1000,900])
print(np.exp(a)/np.sum(np.exp(a)))

c=np.max(a)
print(np.exp(a-c)/np.sum(np.exp(a-c)))
```

c를 안빼줄 경우는 오버플로 발생

c를 빼준경우 결과값 출력

```
[nan nan nan]
[9.99954602e-01 4.53978687e-05 1.68883521e-48]
```

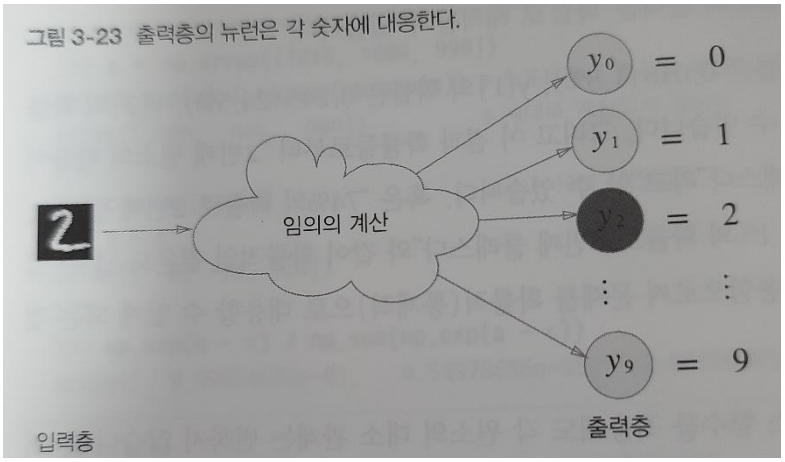
```python
def softmax(a):
    c=np.max(a)
    exp_a=np.exp(a-c)
    sum_exp_a=np.sum(exp_a)
    y=exp_a/sum_exp_a

    return y
```

왼쪽은 오버플로를 고려한 소프트맥스함수다.

(오버플로 :컴퓨터가 표현할수 있는 범위가 한정되어 너무 큰 값은 표현할수 없어서 발생하는 문제)

- 출력층의 뉴런 수 정하기

예를 들어 입력 이미지를 숫자 0부터 9 중 하날 분류하는 문제라면 출력층의 뉴런을 10개로 설정해야 한다. 그 중 출력 값의 크기가 가장 큰것을 선택한다.



색이 가장 짙은 y2 ,뉴런이 가장 큰 값을 출력한다. 그래서 이 신경망이 선택한 클래스는 y2 즉 입력 이미지 숫자 2로 판단했음을 의미힌다.

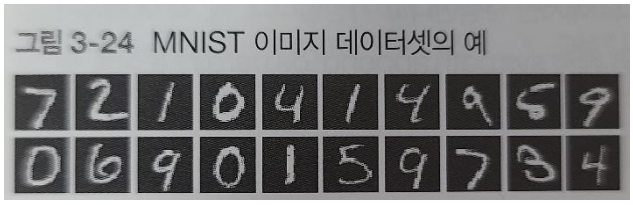
신경망 안에는 각각의 층들이 있는데 그 층마다 w와 b 값이 다르다 그 값에 따라 앞에서 봤던 것처럼 w와 b 값에 따라 AND OR NAND 가 구분되는거처럼 각각의 층마다 어떤 w 와 b 값을 적용하느냐에 따라 그 모델이 달라지게 되는것이다. 그러므로 결과값이 주어진 데이터를 통해 학습을 시키는 것이다.

6. 손글씨 숫자 인식

이미 학습된 매개변수를 사용하여 학습 과정은 생략하고 추론 과정만 구현할것이다. 이 추론 과정을 신경망의 순전파(forward propagation)라고도 한다.

(기계학습과 마찬가지로 신경망도 두 단계를 거쳐 문제를 해결한다. 먼저 훈련 데이터(학습 데이터)를 사용해 가중치 매개변수를 학습하고, 추론 단계에서는 앞서 학습한 매개변수를 사용하여 입력 데이터를 분류한다.)

- MNIST 데이터셋



MNIST 데이터셋은 0부터 9까지의 숫자 이미지로 구성된다. 훈련 이미지가 60,000장 , 시험 이미지가 10,000장 준비되어 있다.

MNIST 이미지의 데이터는 28x28 크기의 회색조 이미지(1채널)이며 각 픽셀은 0에서 255까지의 값을 취한다. 각 이미지에는 '7','2','1' 과 같이 그 이미지가 실제 의미하는 숫자가 레이블로 붙어 있다.

mnist 데이터 가져오기

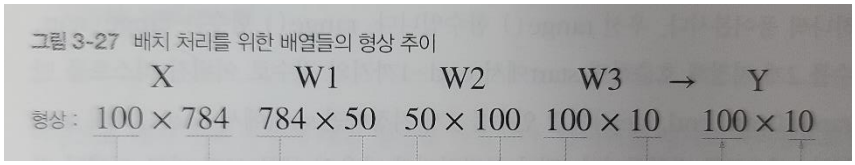
```
# import sys,os
# sys.path.append(os.pardir)
# 부모 디렉터의 파일을 가져올수 있는 방법 dataset 디렉토리가 현재 디렉토리 안에 있는게 아니라
# 부모 디렉토리에 있으면은 위와 같은 코드로 부모 디렉토리의 파일을 가져올 수 있도록 설정한다.
from dataset.mnist import load_mnist
# dataset 폴더에 있는 mnist 모듈로 부터 load_mnist 를 사용하겠다.
(x_train,t_train),(x_test,t_test)=load_mnist(flatten=True,normalize=False,one_hot_label=False)
# flatten 이 True 면 1x28x28 의 3차원 배열을 784개의 원소로 이뤄진 1차원 배열로 저장한다. False 면 1x28x28인 3차원 배열
# normalize 는 0~1 사이 값으로 정규화할지를 정한다. True 면 0~1 값으로 바꿔준다 . False 면 0~255 값 유지
# one_hot_label 이 True 면 숫자를 0과 1로 표현된 배열로 바꿔준다. 7-->[0,0,0,0,0,0,0,1,0,0] False 면 7 그대로 나온다.
print(x_train.shape)
print(t_train.shape)
print(x_test.shape)
print(t_test.shape)
```

입력층 뉴런 784개 출력층 뉴런 10개로 구성된다 은닉층은 총 2개로 구성 첫번째 은닉층은 50개 뉴런 두번째 은닉층은 100개의 뉴런을 배치한다. (50과 100은 임의로 정한값 은닉층 개수도 임의로 정했다.)

```
def predict(network,x):
    W1,W2,W3=network['W1'],network['W2'],network['W3']
    b1,b2,b3=network['b1'],network['b2'],network['b3']

    a1=np.dot(x,W1)+b1
    z1=sigmoid(a1)
    a2=np.dot(z1,W2)+b2
    z2=sigmoid(a2)
    a3=np.dot(z2,W3)+b3
    y=softmax(a3)

    return y
x,t=get_data()
network=init_network()
# 가중치와 편향 초기값

accuracy_cnt=0
for i in range(len(x)): # len(x) 60000개
    y=predict(network,x[i]) # x 데이터 한개 784개의 입력값을 넣어준다.
    p=np.argmax(y) # 확률이 가장 높은 원소의 인덱스를 얻는다.
    if p==t[i]: # y데이터 한개(t[i]) 와 확률이 가장높은 원소의 값이 같다면 정확도를 1을 올린다.( 이미 학습이 다되어있다)
        accuracy_cnt+=1

print("Accuracy:"+str(float(accuracy_cnt)/len(x)))
```

MNIST 정확도 판단과정 -->ch3_10 참고

z1 은 X와 W1으로 만들어진 크기고 z2는 Z1과 W2에 의해 만들어진것이므로 결국은 X W1 W2 로 만들어 진것이다. 그러므로 X W1 W2 W3 차원을 맞추면 된다.



초기값 데이터 개수와 은닉층 노드의 각각 개수 출력값의 분류 개수를 통하여 배열 형상 생성

100장의 입력데이터가 한번에 출력시킬수 있다. (행렬을 통해) 이처럼 하나로 묶은 입력 데이터를 배치(batch)라 한다. 배치가 곧 묶음이다 이미지가 지폐처럼 다발로 묶여 있다고 생각.

(배치 처리를 수행함으로써 큰 배열로 이뤄진 계산을 하게되는데 컴퓨터에서는 큰 배열을 한꺼번에 계산하는 것이 분할된 작은 배열을 여러 번 계싼하는 것보다 빠르다.)

```
x,t=get_data()
network=init_network()
# 가중치와 편향 초기값
batch_size=100
accuracy_cnt=0
for i in range(0,len(x),batch_size): # len(x) 60000개
    x_batch=x[i:i+batch_size]
    y_batch=predict(network,x_batch) # x 데이터 batch사이즈만큼의 행과 784개의열의 입력값을 넣어준다.
    p=np.argmax(y_batch,axis=1) # 확률이 가장 높은 원소의 인덱스를 batch사이즈만큼의 행으로 얻게된다.
    accuracy_cnt+=np.sum(p==t[i:i+batch_size])

    # y데이터 batch사이즈만큼데이터(t[i:i+batch_size])각각 행과 확률이 가장높은 원소의 값이 같다면 True를 리스트에 저장 아니면 False를 저장
    # print(p==t[i:i+batch_size])
    # p==t[i:i+batch_size] batch 사이즈 만큼 리스트에 각각 넘파이 배열끼리 비교하여 True/False로 구성된 bool 배열을 만단다.
    # np.sum(list)에서 list가 True와 False 로 이루어져있다면 True 값만큼 다 더하게 된다.

print("Accuracy:"+str(float(accuracy_cnt)/len(x)))
```

결과는 똑같이 나오게 된다. 단지 100개씩 한번에 계산하므로 계산과정이 조금더 빨라질뿐이다.

Chapter 4 신경망 학습

1. 데이터에서 학습한다.

- 신경망의 특징은 데이터를 보고 학습할 수 있다는 점이다. 데이터에서 학습한다는 것은 가중치 매개변수의 값을 데이터를 보고 자동으로 결정한다는 것이다. 사람이 일일히 모든 매개변수를 수작업으로 결정해야한다고 하면 시간도 엄청나게 걸리고 불가능에 가깝다.

- 기계학습 문제는 데이터를 훈련데이터와 시험 데이터로 나눠 학습과 실험을 수행하는 것이 일반적이다.

- 데이터셋 하나로만 매개변수의 학습과 평가를 수행하면 올바른 평가가 될수 없다. 수중의 데이터셋은 제대로 맞히더라도 다른 데이터셋에는 엉망인 일이 발생할수 있다. 참고로 한 데이터셋에서만 지나치게 최적화된 상태를 오퍼피팅(overfittin_ 이라고 한다. 오버피팅 피하기는 기계학습의 중요한 과제이다.

2. 손실함수(loss function) 비용 함수(cost function)이라고도 한다.)

- 신경망 학습에서는 현재의 상태를 하나의 지표로 표현한다. 이 지표를 가장 좋게 만들어주는 가중치 매개변수의 값을 탐색하는것이다. 신경망 학습에서 사용하는 지표는 손실함수(loss function)이라고 한다. 손실함수에는 임의의 함수를 사용할 수도 있지만 일반적으로는 오차제곱합과 교체엔트로피 오차를 사용한다.

- 오차제곱합(sum of squares for error, SSE)

$$E = \frac{1}{2}\sum_{k}(y_k - t_k)^2$$

y_k: *신경망의 출력(신경망이 추정한 값)*

t_k: *정답 레이블*

k: 데이터의 차원 수

```
y=[0.1,0.05,0.6,0.0,1,0.05,0.1,0.0,0.1,0.0,0.0]
t=[0,0,1,0,0,0,0,0,0,0]
```

t는 숫자 2를 나타낸다.

이처럼 한 원소만 1로 하고 그 외는 0으로 나타내는 표기법을 원-핫인코딩 이라고 한다.

- 오차 제곱합 구현 코드

```python
import numpy as np

# 정답이 2인 경우다 (one-hot encoding) (실제값 )
t=[0,0,1,0,0,0,0,0,0,0]

# 각 원소 추정값 2인경우가 0.6으로 가장 확률이 높다. (출력값 )
y=[0.1,0.05,0.6,0.0,0.05,0.1,0.0,0.1,0.0,0.0]

def sum_squares_error(y,t):
    return 0.5*np.sum((y-t)**2)

# 2일 확률이 가장 높다고 추정한 경우(0.6)
print(sum_squares_error(np.array(y),np.array(t)))
# sum_squares_error 값이 0.0975 로 상당히 낮은 값이 나왔다.

# 7일 확률이 가장 높다고 추정한 경우(0.6)
y=[0.1,0.05,0.1,0.0,0.05,0.1,0.0,0.6,0.0,0.0]
print(sum_squares_error(np.array(y),np.array(t)))
# sum_squares_error 값이 0.5975 로 높은 값이 나왔다.
```

이처럼 손실함수중 하나인 오차제곱합이 작을수록 정확하다는 것이다.

- 교차 엔트로피 오차(cross entropy error ,CEE)

$$E = -\sum_k t_k \log y_k$$

y_k: *신경망의 출력*(*신경망이 추정한 값*)

t_k: *정답 레이블*

k: 데이터의 차원 수

t_k 는 정답에 해당하는 인덱스의 원소만 1이고 나머지는 0이다 (원-핫 인코딩)

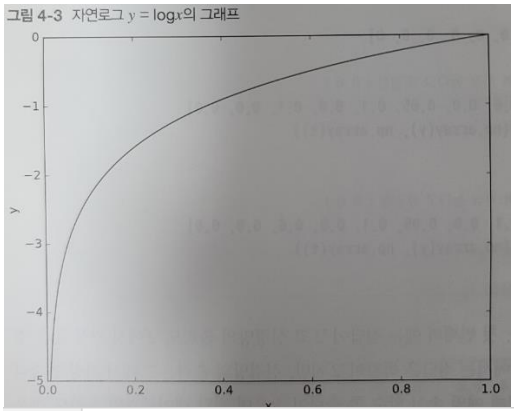
그래서 이식을 실질적으로 정답일때의 추정(t_k 가 1일때의 y_k)의 자연로그를 계싼하는 식이 된다.

이 식에서 log 는 밑이 e 인 자연로그다 (ln)

예를 들어 정답 레이블이 2가 정답일 때 신경망의 출력이 0.6 이라면 교차 엔트로피 오차는 -ln(0.6)=0.51이 되고 같은 조건으로 신경망 출력이 0.1인경우 -ln(0.1)=2.3 가 된다.

즉 교차 엔트로피 오차는 정답일 때의 출력이 전체 값을 정하게 된다.

0부터 1까지의 y=ln(x) 의 그래프를 그려보면 다음과 같다.



x가 1일때 y는 0

x가 0으로 갈수록 y값은 점점 작아지게 된다. 그러므로 -를 붙인값은 점점 커지게 된다.

```python
import numpy as np

def cross_entropy_error(y,t):
    delta=1e-7
    return -np.sum(t*np.log(y+delta))
# log안에 값이 0이 되면 무한대가 되어 더이상 계산을 할수 없기 때문에 -무한대를 방지하기 위하여 delta를 더해줬다.

t=[0,0,1,0,0,0,0,0,0,0]
y=[0.1,0.05,0.6,0.0,0.05,0.1,0.0,0.1,0.0,0.0]

print(cross_entropy_error(np.array(y),np.array(t)))
# 0.51082
y=[0.1,0.05,0.1,0.0,0.05,0.1,0.0,0.6,0.0,0.0]
print(cross_entropy_error(np.array(y),np.array(t)))
# 2.30258
```

cross entropy error 또한 sum of squares for error 와 마찬가지로 값이 정답일수록 값이 더작다.

- 미니배치 학습

기계학습 문제는 훈련 데이터를 사용해 학습한다. 훈련데이터에 대한 손실 하수의 값을 구하고 그 값을 최대한 줄여주는 매개변수를 찾아낸다. 모든 훈련 데이터를 대상으로 손실 함수 값을 구해야 한다. 즉 훈련 데이터가 100개 있으면 그로부터 계산한 100개의 손실 함수 값들의 합을 지표로 삼는 것이다. 데이터가 N개 라면 위에서 배운 SSE나 CEE 로 구한값을 모두 더한다음에 마지막에 N으로 나누어 주면 된다. N으로 나눔으로써 평균 손실 함수를 구하는 것이다. 훈련데이터가 1000개든 10000개든 상관없이 평균 손실 함수를 구할수 있다.

하지만 MNIST 만 해도 훈련데이터가 60000개 인데 모든 데이터를 대상으로 손실 함수의 합을 구하려면 시간이 걸린다. 더나아가 빅데이터 수준이 되면 그수는 너무나도 거대한 값이 되어 일일히 손실함수를 계산하는 것은 현실적이지 않다. 이런 경우 데이터 일부를 추려 전체의 근사치로 이용할수 있는데 신경망 학습에서도 훈련 데이터로부터 일부만 골라 학습을 수행한다. 이 일부를 미니배치(mini-batch)라고 한다. 가령 60000장의 훈련 데이터중 100장을 무작위로 뽑아 그 100장만을 사용하여 학습하는것이다. 이러한 학습방법을 미니배치 학습이라고 한다.

```python
import numpy as np
from dataset.mnist import load_mnist

(x_train,t_train),(x_test,t_test)=load_mnist(flatten=True,normalize=True,one_hot_label=True)
print(x_train.shape)
print(t_train.shape)

train_size=x_train.shape[0]
batch_size=10
batch_mask=np.random.choice(train_size,batch_size)
# np.random.choice(60000,10) 이라 하면 0이상 60000미만 수중에서 무작위로 10개를 골라내는 것이다.
# train_size 60000개 중에 10개를 무작위로 골라낸것(10개 사진 골라내기 255 개 데이터로 표현된 사진 10장)(one_hot encoding 으로 표현된 결과값 10개)
print(batch_mask)
x_batch=x_train[batch_mask]
t_batch=t_train[batch_mask]
print(x_batch)
print(t_batch)
```

(ch4_3)

```
(60000, 784)
(60000, 10)
[30301  6493 35613 50182  1606 45876 40343 44452 10545  1995]
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]
```

```
[[0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```